

Aula 7 Parte 2

Modelos Preditivos: Classificação Conceitos e Fronteiras de Decisão

Objetivos desta sessão

- Retomar a distinção classificação × regressão (alvo categórico × numérico), fechando o ciclo aberto na Aula 6.
- Apresentar 4 novas famílias de classificadores regressão logística, árvore de decisão / random forest, Naive Bayes e SVM além do k-NN, já conhecido desde a Aula 1.
- Introduzir o conceito de fronteira de decisão como forma unificada de comparar essas famílias.
- Reconhecer, em cada família, a ideia central (vizinhança, probabilidade linear, splits, Bayes, margem) sem entrar no treinamento formal.
- Observar visualmente como fronteiras bem diferentes surgem do mesmo conjunto de dados.

1. De regressão para classificação (retomada)

Na Aula 3.1 já dividimos o aprendizado supervisionado em duas variantes pelo tipo do alvo (y): classificação, quando y é uma categoria, e regressão, quando y é um número contínuo. Praticamos classificação desde a Aula 1 (k-NN no Iris) e regressão só ontem, na Aula 7.1 (aluguéis). Hoje fechamos o ciclo: voltamos para classificação, mas com o vocabulário mais rico que ganhamos no caminho sobretudo o conceito de fronteira/margem que apareceu com o SVR na Aula 6.

A pergunta que separa os dois continua a mesma: se eu embaralhar os valores de y, a ordem importa? Em classificação, não "aprovado" não é "maior" que "não aprovado", são só rótulos diferentes. Isso vale para todos os algoritmos de hoje: nenhum deles assume qualquer ordem entre as classes.

2. Fronteira de decisão: a ideia central de hoje

Um classificador divide o espaço dos atributos em regiões uma para cada classe. A fronteira de decisão é a linha (ou curva, ou superfície, em mais de 2 dimensões) que separa essas regiões: de um lado, o modelo prevê uma classe; do outro, a classe oposta. Você já usa essa ideia sem nome desde a Aula 1 o k-NN, ao votar a maioria dos k vizinhos mais próximos, também traça uma fronteira, só que uma fronteira irregular, cheia de "dentes", porque ela reage a cada ponto individual de treino.

A partir de hoje, cada nova família que vemos pode ser resumida por DUAS perguntas: (1) qual é a ideia central que ela usa para decidir uma classe? e (2) que FORMATO de fronteira essa ideia produz reta, curva suave, escadinha de retângulos, ou algo irregular como o k-NN? É esse formato que vamos comparar visualmente na Seção 7.

2.1. k-NN retomada rápida

Ideia central: olha para os k vizinhos mais próximos (por distância, Aula 5) e vota a maioria. Não existe fase de "treinamento" com ajuste de coeficientes o k-NN memoriza os dados de treino e só faz o trabalho no momento de prever (por isso é chamado de método "lazy", ou baseado em instância). Fronteira tipicamente irregular, com "dentes" que acompanham de perto os pontos de treino quanto menor o k, mais irregular (Aula 1).

3. Regressão logística

Apesar do nome, regressão logística é um algoritmo de CLASSIFICAÇÃO o nome vem de como ele é formulado, não do tipo de problema que resolve. A ideia: começa com uma combinação linear dos atributos, exatamente como na regressão linear (Aula 6/7.1) —

$$z = b_0 + b_1x_1 + b_2x_2 + \dots + b_px_p$$

— mas em vez de usar z diretamente como previsão, passa z por uma função sigmoide, que "espreme" qualquer número real para o intervalo (0, 1):

$$p = 1 / (1 + e^{(-z)}) \quad \# p = \text{probabilidade estimada da classe 1}$$

p é interpretado como a probabilidade estimada de a classe ser 1; se $p > 0,5$, o modelo prevê a classe 1, senão prevê a classe 0. Como o corte ($p = 0,5$) corresponde a $z = 0$ uma equação LINEAR nos atributos a fronteira de decisão da regressão logística é, no fundo, uma reta (ou um hiperplano, com mais atributos). É o mesmo tipo de limitação e vantagem que a regressão linear tem: simples, rápida e fácil de interpretar (cada bi ainda diz "quanto" aquele atributo empurra a probabilidade para cima ou para baixo), mas só separa bem classes que são aproximadamente separáveis por uma reta.

4. Árvore de decisão e random forest

Ideia central: uma sequência de perguntas do tipo "o atributo X é maior que o valor V?", organizadas em uma estrutura de árvore. Cada pergunta divide os dados em dois grupos mais "puros" (com menos mistura de classes) do que o grupo anterior; a árvore aprende, nos dados de treino, QUAL atributo e QUAL valor de corte usar em cada pergunta, para maximizar essa separação. Uma folha da árvore corresponde a uma classe prevista.

Porque cada pergunta corta o espaço em uma linha reta perpendicular a um dos eixos (" $x_1 > 5$ ", nunca " $x_1 + x_2 > 5$ "), a fronteira final de uma árvore é sempre uma escadinha de retângulos nunca uma curva suave ou uma diagonal genuína. Árvores profundas encaixam os dados de treino muito de perto (risco de overfitting, o mesmo problema já visto na regressão polinomial de grau alto, Aula 6); é por isso que hoje limitamos a profundidade ($\text{max_depth}=4$) na demonstração.

Random forest, em uma frase

Random forest treina MUITAS árvores de decisão diferentes (cada uma vendo uma amostra ligeiramente diferente dos dados e dos atributos) e faz elas votarem juntas. O objetivo é reduzir o overfitting de uma árvore única: erros individuais de árvores distintas tendem a se cancelar na votação, e a fronteira final fica mais suave e mais estável do que a de uma árvore isolada sem perder a lógica de "perguntas sequenciais" que já conhecemos. Não praticamos random forest isoladamente hoje, mas ele reaparece no laboratório comparativo da Semana 8 (21/09).

5. Naive Bayes

Ideia central: usa o teorema de Bayes para estimar, para cada classe, a probabilidade de observar os atributos de um exemplo DADO que ele pertence àquela classe e escolhe a classe com maior probabilidade estimada. O termo "naive" (ingênuo) vem de uma simplificação: o modelo assume que os atributos são INDEPENDENTES entre si, dado a classe (por exemplo, que renda e tempo_emprego não têm relação alguma um com o outro, uma vez que já sabemos se a pessoa foi aprovada ou não) uma suposição quase sempre falsa na prática, mas que na prática funciona surpreendentemente bem e torna o cálculo muito rápido, mesmo com muitos atributos.

Na versão Gaussiana (GaussianNB, usada hoje), cada atributo numérico é modelado, dentro de cada classe, por uma distribuição normal (curva de sino) por isso a fronteira de decisão do Naive Bayes costuma ser uma curva suave, sem os "dentes" do k-NN nem os "cantos" da árvore.

6. SVM retomando a margem do SVR

Na Aula 6 vimos o SVR (Support Vector Regression) ajustando um "tubo" de tolerância ao redor de uma função contínua. O SVM (Support Vector Machine) usa exatamente a mesma ideia de margem só que para CLASSIFICAÇÃO, não regressão: em vez de um tubo ao redor de uma curva, o SVM busca a fronteira que deixa a MAIOR margem (distância) possível até os pontos mais próximos de cada classe. Esses pontos mais próximos os que realmente definem onde fica a margem são os vetores de suporte; a maioria dos pontos "fáceis", bem distantes da fronteira, não influencia o resultado.

Com $\text{kernel}=\text{"linear"}$, a fronteira do SVM é uma reta (como a da regressão logística). Com $\text{kernel}=\text{"rbf"}$ (usado na demonstração de hoje), o SVM projeta os dados para um espaço onde ficam mais fáceis de separar, e a fronteira final, de volta no espaço original, pode ser uma curva bem flexível capaz de acompanhar formatos como o das duas "luas" da nossa demonstração.

7. Comparação visual: 5 famílias, mesmo dataset

Para deixar as diferenças de formato bem visíveis, a demonstração de hoje usa um dataset SINTÉTICO (make_moons) dois grupos em forma de "lua" entrelaçada, escolhidos de propósito porque NÃO podem ser separados por uma única reta. O dataset real e barulhento das solicitações de empréstimo (Aulas 4/5) volta na Semana 8, quando comparamos os mesmos 5 algoritmos com validação cruzada formal.

Mesmos dados, 5 fronteiras de decisão diferentes (pontos = conjunto de teste)

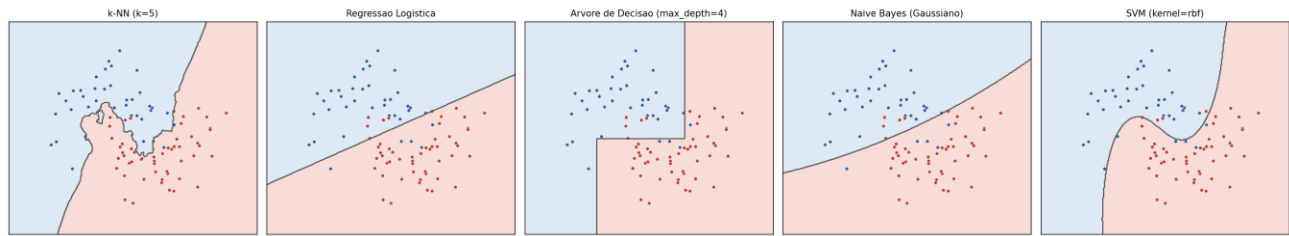


Figura 1 as mesmas 210 observações de treino, 5 fronteiras de decisão diferentes. Pontos azuis e vermelhos são o conjunto de teste (nunca visto no treino).

Repare no formato: o k-NN ($k=5$) produz uma fronteira irregular, cheia de "reentrâncias", porque reage à posição exata de cada vizinho. A regressão logística traça (aproximadamente) uma reta é a família mais restrita das cinco. A árvore de decisão corta o espaço em blocos retangulares (a "escadinha" da Seção 4). Naive Bayes produz uma curva suave, sem cantos. O SVM com kernel rbf também produz uma curva suave, mas mais flexível consegue acompanhar melhor o formato das duas "luas" do que a regressão logística, sem os cantos da árvore.

No conjunto de teste da demonstração, um ponto específico perto da fronteira entre as duas luas foi classificado como 1 por quatro dos cinco modelos, mas a árvore de decisão (com $\text{max_depth}=4$) o classificou como 0. Isso não significa que a árvore "errou" significa que, exatamente naquela região, o corte retangular da árvore ficou posicionado de um jeito ligeiramente diferente das curvas suaves dos outros modelos. É um exemplo concreto de por que comparar apenas a acurácia total não conta toda a história: dois modelos com acurácia parecida podem discordar bastante em pontos específicos, perto da fronteira.

Um cuidado importante

Nenhuma dessas 5 famílias é "a melhor" de forma absoluta. Regressão logística e SVM linear funcionam bem quando as classes são aproximadamente separáveis por uma reta; árvore de decisão lida bem com interações do tipo "se X e também Y"; Naive Bayes é rápido e funciona surpreendentemente bem mesmo quando a suposição de independência é violada; k-NN e SVM com kernel não linear lidam bem com fronteiras irregulares, ao custo de mais tempo de previsão (k-NN) ou mais hiperparâmetros para ajustar (SVM). A comparação formal com validação cruzada, no dataset real é o assunto da Semana 8 (21/09) e da Semana 9 (28/09).

O que guardar desta sessão

- Regressão logística é classificação, apesar do nome usa uma combinação linear passada por uma sigmoide; a fronteira final é (aproximadamente) uma reta.
- Árvore de decisão corta o espaço com perguntas do tipo "atributo > valor?" a fronteira é sempre uma escadinha de retângulos; random forest treina várias árvores e faz elas votarem, reduzindo o overfitting de uma árvore única.
- Naive Bayes usa o teorema de Bayes assumindo (de forma simplificadora) que os atributos são independentes entre si, dado a classe rápido e surpreendentemente eficaz na prática.
- SVM retoma a ideia de margem do SVR (Aula 6): busca a fronteira com a maior distância possível até os pontos mais próximos de cada classe (os vetores de suporte); com kernel não linear, a fronteira pode virar uma curva flexível.
- Fronteiras diferentes surgem do MESMO dataset comparar apenas a acurácia total esconde que os modelos podem discordar bastante perto da fronteira, mesmo com desempenho global parecido.